NyumbaPMS — Project Context & Development Roadmap

> **Purpose:** This document is the master context for the NyumbaPMS project. It records what the project is, what has been completed, important architectural decisions, the current development state, and what remains to be built.
>
> **Important:** Update this document whenever a major feature, architectural decision, database change, deployment change, or development milestone is completed.

---

1. PROJECT IDENTITY

Project Name

NyumbaPMS

Official project name: NyumbaPMS.

Project Type

Kenyan Property Management SaaS / Property Operating System.

Target Market

Kenya.

Primary Users

- Individual landlords
- Property managers
- Property management companies
- Real estate businesses
- Property owners with multiple rental units
- Property management staff
- Caretakers
- Accountants

Main Goal

Build a professional, scalable, monetizable property-management platform designed specifically for the Kenyan rental market.

The product should not be just a simple rent-recording application.

The long-term goal is to build a **Property Operating System for Kenya** that handles the major operational, financial, communication, and management activities involved in running rental properties.

---

2. CORE PRODUCT VISION

NyumbaPMS should allow a landlord or property manager to manage their rental business from one platform.

The platform should eventually cover:

- Properties
- Buildings
- Units
- Tenants
- Leases
- Rent billing
- Payments
- M-Pesa
- Arrears
- Receipts
- Notifications
- WhatsApp/SMS communication
- Maintenance
- Expenses
- Utilities
- Accounting
- Documents
- Reports
- Analytics
- AI assistance
- Staff management
- Subscriptions
- Audit trails
- Support

The system should be designed as a serious SaaS rather than a basic CRUD application.

---

3. CORE BUSINESS FLOW

The fundamental relationship is:

```
USER / ORGANIZATION
        ↓
PROPERTY
        ↓
BUILDING
        ↓
UNIT
        ↓
LEASE
        ↓
TENANT
        ↓
BILLING
        ↓
PAYMENT
        ↓
M-PESA
```

Example:

```
Landlord
   ↓
Apartment Property
   ↓
Building A
   ↓
Unit A-101
   ↓
Tenant
   ↓
Monthly Lease
   ↓
Rent Invoice
   ↓
Tenant Pays
   ↓
M-Pesa
   ↓
Payment Callback
   ↓
System Reconciles Payment
   ↓
Tenant Balance Updated
   ↓
Receipt Generated
```

---

4. PRODUCT PRINCIPLES

The application should be:

- Professional
- Secure
- Scalable
- Mobile-first
- Responsive
- Easy to use
- Kenyan-market focused
- API-ready
- SaaS-ready
- Financially reliable
- Auditable
- Modular
- Maintainable

The UI should work particularly well on mobile phones because many Kenyan landlords, caretakers, and tenants will access the system using mobile devices.

---

5. TECHNOLOGY STACK

Backend

- Python
- Django

Database

Development

SQLite is currently being used temporarily.

Planned

PostgreSQL.

PostgreSQL will be the serious database for the production system.

The PostgreSQL setup/password issue is currently being postponed and should be handled later.

Frontend

Planned:

- HTML

- CSS
- JavaScript
- Bootstrap
- Flexbox
- Responsive/mobile-first design

Charts / Analytics

Planned:

- Chart.js

PDF

Planned:

- ReportLab

Payments

Planned:

- M-Pesa / Safaricom Daraja API

Notifications

Planned:

- WhatsApp
- SMS
- Email
- In-app notifications

Environment Variables

Planned/current approach:

```
.env
python-dotenv
```

Sensitive credentials must never be hard-coded into source code.

---

6. DJANGO PROJECT STRUCTURE

Current project root:

```
landlordpms/
```

Current main structure:

```
landlordpms/
│
├── config/
│
├── accounts/
├── properties/
├── tenants/
├── leases/
├── billing/
├── payments/
├── mpesa/
├── notifications/
├── maintenance/
├── expenses/
├── accounting/
├── utilities/
├── documents/
├── reports/
├── analytics/
├── ai_assistant/
├── subscriptions/
├── audit/
├── support/
│
├── templates/
│   └── base/
│
├── static/
│   ├── css/
│   ├── js/
│   └── images/
│
├── media/
│
├── manage.py
├── db.sqlite3
├── requirements.txt
├── .env
└── .gitignore
```

---

7. DJANGO APPS

The planned application architecture contains the following apps.

Core

1. `accounts`

2. `properties`
3. `tenants`
4. `leases`
5. `billing`
6. `payments`
7. `mpesa`
8. `notifications`

Business Operations

9. `maintenance`
10. `expenses`
11. `accounting`
12. `utilities`
13. `documents`
14. `reports`
15. `analytics`

Advanced

16. `ai_assistant`
17. `subscriptions`
18. `audit`
19. `support`

These apps are intentionally separated so the project can grow into a large SaaS without putting everything into one Django application.

---

8. MVP SCOPE

The first commercial MVP should NOT attempt to build every planned feature simultaneously.

The initial MVP should focus on:

```
accounts
properties
tenants
leases
billing
payments
mpesa
notifications
```

These form the main rental-management engine.

After the core system works reliably, additional modules can be introduced.

---

9. FUTURE MODULES

After the MVP:

Maintenance

- Maintenance requests
- Work orders
- Maintenance status
- Contractors
- Maintenance costs
- Tenant maintenance reporting

Expenses

- Property expenses
- Expense categories
- Receipts
- Expense tracking

Accounting

- Income
- Expenses
- Financial summaries
- Profit/loss
- Reconciliation
- Financial reports

Utilities

- Water
- Electricity
- Shared utilities
- Utility billing
- Meter readings

Documents

- Lease documents
- Tenant documents
- Property documents
- Receipts
- Contracts
- File storage

Reports

- Rent collection reports

- Arrears reports
- Occupancy reports
- Income reports
- Expense reports
- Property performance

Analytics

- Occupancy rate
- Collection rate
- Arrears trends
- Revenue trends
- Property performance
- Tenant statistics

AI Assistant

Potential future functionality:

- Property performance insights
- Rent collection analysis
- Arrears predictions
- Automated summaries
- Business questions
- Recommendations
- Anomaly detection

Subscriptions

- SaaS plans
- Unit limits
- Billing
- Subscription status
- Feature restrictions
- Upgrade/downgrade

Audit

Track important actions such as:

- Login
- Property changes
- Tenant changes
- Lease changes
- Billing changes
- Payment changes
- Staff actions
- Administrative actions

Support

- Support tickets
- Help center
- Customer support
- Issue tracking

---

10. USER / ACCOUNT ARCHITECTURE

This is one of the most important architectural areas.

A custom Django User model should be created **before migrations and before other models begin depending heavily on the user model**.

Potential roles:

```
OWNER / LANDLORD
PROPERTY_MANAGER
ACCOUNTANT
CARETAKER
STAFF
ADMIN
```

The system should support the possibility that:

- One landlord owns multiple properties.
- One property manager can manage multiple properties.
- A management company can have multiple staff members.
- Staff members can have different permissions.
- Users may work with different properties.
- A user should only see data they are authorized to access.

The final organization/ownership architecture must be designed carefully before building the main database models.

---

11. MULTI-TENANCY / SAAS ARCHITECTURE

NyumbaPMS is intended to become a SaaS.

Therefore, the architecture must consider:

```
Organization / Account
        ↓
Users
        ↓
Properties
        ↓
Buildings
        ↓
Units
        ↓
Tenants
        ↓
Leases
        ↓
Billing
        ↓
Payments
```

The system must prevent one landlord/company from accessing another landlord/company's data.

Data isolation and authorization are critical.

---

12. PROPERTY STRUCTURE

Expected hierarchy:

```
Property
    ↓
Building
    ↓
Unit
```

A property could represent:

- Apartment complex
- Residential estate
- Commercial building
- Mixed-use property
- Single rental building
- Multiple-unit property

Each unit should eventually contain information such as:

- Unit number
- Unit type
- Rent amount
- Status
- Floor

- Property
- Occupancy
- Utility information

---

13. TENANT STRUCTURE

Tenants should eventually have information such as:

- Name
- Phone number
- Email
- Identification information
- Emergency contact
- Lease history
- Current lease
- Payment history
- Outstanding balance
- Documents

Tenant data must be protected carefully.

---

14. LEASE STRUCTURE

A lease connects a tenant to a unit.

Expected information:

- Tenant
- Unit
- Start date
- End date
- Rent amount
- Deposit
- Billing frequency
- Lease status
- Terms
- Documents

Possible lease statuses:

```
ACTIVE
EXPIRED
TERMINATED
PENDING
```

---

15. BILLING SYSTEM

The billing system should eventually support:

- Monthly rent
- Deposits
- Utility charges
- Other charges
- Discounts
- Adjustments
- Due dates
- Invoices
- Balances
- Arrears

A tenant's financial state should be calculated reliably.

Example:

```
Opening Balance
+ New Charges
- Payments
= Outstanding Balance
```

The billing engine must avoid duplicate charges and duplicate transactions.

---

16. PAYMENT SYSTEM

Payments should eventually support:

- M-Pesa
- Manual payments
- Bank payments
- Other payment methods

Payment records should include:

- Amount
- Tenant
- Lease
- Invoice
- Payment date
- Reference
- Payment method
- Status
- Transaction ID

Payments must be auditable.

---

17. M-PESA INTEGRATION

The intended payment flow:

```
Tenant
   ↓
Rent Payment
   ↓
M-Pesa
   ↓
Safaricom
   ↓
Callback
   ↓
NyumbaPMS
   ↓
Identify Tenant
   ↓
Record Payment
   ↓
Reconcile Invoice
   ↓
Update Balance
   ↓
Generate Receipt
   ↓
Notify Tenant / Landlord
```

M-Pesa integration should be implemented only after the underlying payment and billing models are stable.

---

18. NOTIFICATION SYSTEM

The platform should eventually support:

WhatsApp

Examples:

- Rent due reminder
- Payment confirmation
- Receipt
- Arrears reminder
- Maintenance update

- Lease expiry reminder

SMS

For users who do not use WhatsApp or where SMS is preferable.

Email

For:

- Receipts
- Reports
- Account notifications
- Documents
- Administrative messages

In-app

For dashboard notifications.

---

19. SECURITY REQUIREMENTS

Security is a major priority.

The system should eventually include:

- Authentication
- Authorization
- Role-based permissions
- CSRF protection
- Secure password handling
- Environment variables
- Secure file uploads
- Input validation
- Access control
- Audit logs
- Rate limiting where appropriate
- Secure payment callbacks
- Protection against duplicate payment processing
- Tenant/company data isolation

Never expose:

- Secret keys
- API credentials
- M-Pesa credentials
- Database passwords
- Production secrets

in GitHub.

---

20. FILE UPLOAD SECURITY

Documents and uploads must be handled securely.

Potential uploaded files include:

- Lease agreements
- Tenant documents
- Property documents
- Receipts
- Maintenance images

The application must validate:

- File type
- File extension
- File size
- User permissions

Uploaded files should not automatically be treated as executable content.

---

21. CURRENT ENVIRONMENT

Operating system:

```
Windows 11
```

Development environment:

```
VS Code
Command Prompt
Python virtual environment
Django
```

Project location:

```
C:\Users\user\OneDrive\Desktop\webdev2025\landlordpms
```

Virtual environment:

```
venv/
```

---

22. CURRENT SETUP COMPLETED

Project created

The Django project has been created using:

```
python -m django startproject config .
```

This produced:

```
config/
manage.py
```

---

23. VIRTUAL ENVIRONMENT

Created using:

```
python -m venv venv
```

Activation:

```
venv\Scripts\activate
```

The virtual environment is working.

---

24. REQUIREMENTS

Current `requirements.txt`:

```
Django
psycopg[binary]
Pillow
python-dotenv
```

Purpose:

- Django → web framework
- psycopg → PostgreSQL connectivity
- Pillow → image processing
- python-dotenv → environment variables

---

25. ENVIRONMENT VARIABLES

Current `.env` contains:

```
DEBUG=True
SECRET_KEY=change-this-later
```

This is a temporary development configuration.

A real production secret key must eventually be generated and stored securely.

---

26. GITIGNORE

Current `.gitignore` should include:

```
# Virtual environment
venv/
.env

# Python
__pycache__/
*.py[cod]
*$py.class

# Django
db.sqlite3
media/
staticfiles/

# IDE
.vscode/
.idea/

# OS
.DS_Store
Thumbs.db

# Logs
*.log
```

Important:

`.env` and database files must not be pushed to GitHub.

---

27. DJANGO SETTINGS

The settings have been configured to use environment variables.

Important settings include:

```
from pathlib import Path
import os
from dotenv import load_dotenv

load_dotenv()
```

Secret key:

```
SECRET_KEY = os.getenv("SECRET_KEY")
```

Debug:

```
DEBUG = os.getenv("DEBUG", "False").lower() == "true"
```

Templates:

```
TEMPLATES[0]["DIRS"] = [BASE_DIR / "templates"]
```

Static files:

```
STATIC_URL = "static/"
STATICFILES_DIRS = [BASE_DIR / "static"]
STATIC_ROOT = BASE_DIR / "staticfiles"
```

Media:

```
MEDIA_URL = "/media/"
MEDIA_ROOT = BASE_DIR / "media"
```

---

28. DIRECTORIES CREATED

```
templates/
templates/base/

static/
static/css/
static/js/
static/images/

media/
```

These will support the frontend and uploaded files.

---

29. MEDIA DEVELOPMENT CONFIGURATION

Development media serving has been added to `config/urls.py`.

Current approach:

```
from django.conf import settings
from django.conf.urls.static import static

if settings.DEBUG:
    urlpatterns += static(
        settings.MEDIA_URL,
        document_root=settings.MEDIA_ROOT
    )
```

This is for development.

Production media/file storage will need a proper production strategy.

---

30. DJANGO APPS CREATED

The following apps have been created/planned:

```
accounts
properties
tenants
leases
billing
payments
mpesa
notifications
maintenance
expenses
accounting
utilities
documents
reports
analytics
ai_assistant
subscriptions
audit
support
```

---

31. INSTALLED_APPS

The Django settings are intended to contain:

```
INSTALLED_APPS = [
    # Django built-in apps
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",

    # NyumbaPMS — Core
    "accounts",
    "properties",
    "tenants",
    "leases",
    "billing",
    "payments",
    "mpesa",
    "notifications",

    # NyumbaPMS — Business Operations
    "maintenance",
    "expenses",
    "accounting",
    "utilities",
    "documents",
    "reports",
    "analytics",

    # NyumbaPMS — Advanced
    "ai_assistant",
    "subscriptions",
    "audit",
    "support",
]
```

After adding the apps, the project should be checked with:

```
python manage.py check
```

---

32. DATABASE STATUS

Current local database:

```
SQLite
```

File:

```
db.sqlite3
```

PostgreSQL is planned.

The local PostgreSQL command:

```
psql
```

was not available directly from CMD because PostgreSQL's executable is not currently on the PATH.

The PostgreSQL password is also currently unavailable.

This is intentionally postponed.

Do NOT allow the PostgreSQL issue to block application development.

---

33. DATABASE STRATEGY

Development can temporarily continue with SQLite.

Before serious production deployment:

```
SQLite
   ↓
PostgreSQL
```

The production system should use PostgreSQL.

---

34. DEVELOPMENT STATUS

Current approximate status:

```
[✓] Project concept defined
[✓] Kenyan market selected
[✓] SaaS direction selected
[✓] Django project created
[✓] Virtual environment created
[✓] requirements.txt created
[✓] .env created
[✓] .gitignore created
[✓] Templates directories created
[✓] Static directories created
[✓] Media directory created
[✓] Django settings partially configured
[✓] Django apps created/planned
[✓] Apps registered/planned in settings
[ ] Custom User model
[ ] Organization/account architecture
[ ] Roles and permissions
[ ] Core database models
[ ] Database migrations
[ ] Admin configuration
[ ] Authentication UI
[ ] Dashboard
[ ] Property management
[ ] Unit management
[ ] Tenant management
[ ] Lease management
[ ] Billing engine
[ ] Payment system
[ ] M-Pesa integration
[ ] Notifications
[ ] Maintenance
[ ] Expenses
[ ] Accounting
[ ] Utilities
[ ] Documents
[ ] Reports
[ ] Analytics
[ ] AI
[ ] Subscriptions
[ ] Audit system
[ ] Support
[ ] Production deployment
```

---

35. IMMEDIATE NEXT STEP

The next major task is to design and implement the **accounts architecture**.

This must happen before creating the main business models.

Order:

```
1. Custom User
2. Organization / Account
3. Roles
4. Permissions
5. User ↔ Organization relationship
6. Property ownership/management relationship
7. Authentication
8. Admin
```

Do NOT rush into creating every model before this architecture is settled.

---

36. CORE MODEL DEVELOPMENT ORDER

Recommended order:

```
ACCOUNTS
   ↓
PROPERTIES
   ↓
TENANTS
   ↓
LEASES
   ↓
BILLING
   ↓
PAYMENTS
   ↓
MPESA
   ↓
NOTIFICATIONS
```

Then:

```
MAINTENANCE
EXPENSES
ACCOUNTING
UTILITIES
DOCUMENTS
REPORTS
ANALYTICS
```

Then:

```
AI
SUBSCRIPTIONS
AUDIT
SUPPORT
```

---

37. EXPECTED PROPERTY RELATIONSHIPS

Initial conceptual structure:

```
Organization
    │
    ├── Users
    │
    └── Properties
            │
            └── Buildings
                    │
                    └── Units
                            │
                            └── Leases
                                    │
                                    └── Tenants
```

This is conceptual and must be refined during model design.

---

38. IMPORTANT ARCHITECTURAL RULES

Rule 1 — Custom User First

Do not build the rest of the database around Django's default User model if the application requires a custom account architecture.

Rule 2 — Do Not Put Everything in One App

Business domains should remain separated.

Rule 3 — Financial Data Must Be Reliable

Billing and payment records should not be casually deleted.

Use statuses, reversals, adjustments, and audit records where appropriate.

Rule 4 — Payment Processing Must Be Idempotent

The same M-Pesa callback must not create duplicate payments.

Rule 5 — Authorization Must Be Explicit

A user should only access properties and financial records they are permitted to access.

Rule 6 — Never Trust Client Input

Validate important financial and account information server-side.

Rule 7 — Sensitive Data Must Be Protected

Never commit secrets to GitHub.

Rule 8 — Build MVP First

Do not attempt to build all 19 apps at full complexity immediately.

---

39. MVP USER JOURNEY

A typical landlord journey should eventually be:

```
Register
   ↓
Create Organization / Account
   ↓
Create Property
   ↓
Create Building
   ↓
Create Units
   ↓
Add Tenant
   ↓
Create Lease
   ↓
Generate Rent Billing
   ↓
Tenant Pays
   ↓
M-Pesa
   ↓
Payment Reconciled
   ↓
Receipt Generated
   ↓
Tenant Balance Updated
   ↓
Dashboard Updated
```

---

40. LANDLORD DASHBOARD — FUTURE

The main dashboard should eventually show:

- Total properties
- Total units
- Occupied units
- Vacant units
- Expected rent
- Collected rent
- Outstanding rent
- Arrears
- Collection rate
- Recent payments
- Upcoming rent
- Lease expirations
- Maintenance requests
- Expenses
- Revenue trends

Charts can be implemented using Chart.js.

---

41. MOBILE-FIRST REQUIREMENT

The platform should be designed mobile-first.

Important screens should work well on:

- Android phones
- iPhones
- Tablets
- Desktop computers

Do not design desktop-only interfaces and attempt to shrink them later.

---

42. MONETIZATION

NyumbaPMS is intended to become a paid SaaS.

Potential pricing structure discussed:

```
FREE
5 units

STARTER
25 units
~ KSh 799/month

BUSINESS
100 units
~ KSh 1,999/month

PROFESSIONAL
300 units
~ KSh 4,999/month

ENTERPRISE
Custom pricing
```

These prices are preliminary ideas and must be validated against the Kenyan market before launch.

Potential monetization channels:

- Monthly subscriptions
- Annual subscriptions
- Premium features
- Additional units
- WhatsApp/SMS usage
- Advanced analytics
- AI features
- Property management company plans
- Enterprise plans

---

43. COMPETITIVE POSITIONING

The product should compete on more than rent collection.

Potential differentiation:

```
Property Management
+
M-Pesa
+
Automated Communication
+
Billing
+
Arrears
+
Maintenance
+
Accounting
+
Analytics
+
AI
```

The objective is to make NyumbaPMS a complete operating platform for rental businesses.

---

44. FUTURE AI DIRECTION

AI should not be added simply as a chatbot.

Potential useful AI capabilities:

Financial insights

"Which properties are performing poorly?"

Arrears analysis

"Which tenants are consistently late?"

Revenue forecasting

"What could next month's rental collection look like?"

Property insights

"Which property has the highest collection rate?"

Automated summaries

Generate monthly management summaries.

Anomaly detection

Identify unusual payment or expense patterns.

Natural language analytics

Example:

```
"Show me properties with more than 10% vacancy."

"How much rent is currently outstanding?"

"Which tenants have been late three times?"

"What was my total rental income last month?"
```

---

45. DEVELOPMENT WORKFLOW

Development should proceed step-by-step.

For every major stage:

```
Design
   ↓
Implement
   ↓
Check
   ↓
Migrate
   ↓
Test
   ↓
Admin
   ↓
UI
   ↓
Commit
```

Do not build huge amounts of code without testing each stage.

---

46. TESTING STRATEGY

Testing should eventually include:

- Model tests
- Form tests
- View tests
- Permission tests
- Billing tests
- Payment tests

- M-Pesa callback tests
- API tests
- Security tests
- Integration tests

Especially important:

```
Billing
Payment
M-Pesa
Permissions
Data isolation
```

---

47. GIT WORKFLOW

Git should be used throughout development.

Recommended workflow:

```
git status
git add .
git commit -m "Descriptive message"
```

Examples:

```
git commit -m "Set up Django project structure"
git commit -m "Add custom user architecture"
git commit -m "Add property and unit models"
git commit -m "Add tenant and lease management"
git commit -m "Add billing engine"
```

Never commit:

```
.env
db.sqlite3
venv/
```

---

48. PRODUCTION PLAN

Potential production architecture:

```
User
  ↓
Domain
  ↓
Web Application
  ↓
Django
  ↓
PostgreSQL
  ↓
External Services
```

Potential services:

- PostgreSQL
- Hosting platform
- M-Pesa Daraja
- WhatsApp provider
- SMS provider
- Email provider
- File storage
- Monitoring
- Error tracking

Exact providers will be selected later.

---

49. FUTURE DEPLOYMENT REQUIREMENTS

Before production:

- DEBUG=False
- Strong SECRET_KEY
- PostgreSQL
- Secure environment variables
- HTTPS
- Secure cookies
- CSRF configuration
- Allowed hosts
- Static file deployment
- Media/file storage
- Database backups
- Error monitoring
- Logging
- Payment callback security
- Domain
- Production email
- M-Pesa production credentials

50. CURRENT PROJECT RULE

The project should be developed **step by step with ChatGPT**.

When continuing development, first check this document to understand:

1. What has already been completed.
2. What is currently being worked on.
3. What the next architectural step is.
4. What decisions have already been made.
5. What should not be changed without discussion.

Do not unnecessarily recreate already completed work.

---

51. CURRENT NEXT ACTION

NEXT TASK:

Build the `accounts` architecture.

Specifically:

```
Custom User
     ↓
Organization
     ↓
Membership
     ↓
Roles
     ↓
Permissions
```

Then connect this architecture to:

```
Properties
```

Only after this foundation is stable should the main property, tenant, lease, billing, and payment models be created.

---

52. PROJECT STATUS SUMMARY

At the moment, NyumbaPMS is at the:

FOUNDATION / ARCHITECTURE STAGE

Completed:

- Project idea
- Product vision
- Kenyan market focus
- SaaS direction
- Django project
- Environment
- Requirements
- Static/media/templates structure
- Core app structure
- Initial settings

Not yet completed:

- Custom User
- Organizations
- Permissions
- Database architecture
- Core models
- Authentication
- Dashboard
- Billing
- Payments
- M-Pesa
- Notifications
- Production deployment

The project should now proceed from the account architecture into the core property-management engine.

---

53. QUICK CONTEXT FOR FUTURE SESSIONS

If this project is resumed in another conversation, the essential context is:

> **NyumbaPMS is a Kenyan property-management SaaS being built with Django/Python. It is intended to become a complete Property Operating System rather than a simple rent tracker. The core MVP consists of accounts, properties, tenants, leases, billing, payments, M-Pesa and notifications. The project is designed to be multi-tenant, mobile-first, scalable and monetizable. The Django project and app structure have already been created. SQLite is temporarily being used while PostgreSQL setup is postponed. The next major task is designing the custom User + Organization + Membership + Roles/Permissions architecture before building the main business models.**

END OF PROJECT CONTEXT